🦓 Zebraw

Zebraw is a lightweight and fast package for displaying code blocks with line numbers in typst, supporting code line highlighting. The term ***Zebraw*** is a combination of ***zebra*** and ***raw***, for the highlighted lines will be displayed in the code block like a zebra lines.

Contents

🦓 Zebraw 1
- Starting 1
- Features 2
  - Line Highlighting 2
  - Comment 2
  - Header and Footer 2
  - Language Tab 2
  - Copyable 2
- Customization 3
  - Inset 3
  - Colors 3
  - Font 3
  - Extend 3
- Documentation 4
  - zebraw-init 4
  - zebraw 4
- Example 5

Starting

Import `zebraw` package by `#import "src/lib.typ": *` then follow with `#show: zebraw` to start using zebraw in the simplest way. To manually display some specific code blocks in zebraw, you can use `#zebraw()` function:

```
```typ
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

#zebraw(
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)

#show: zebraw

```typ
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```
```

```
#grid(
  columns: (1fr, 1fr),
  [Hello], [world!],
)
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

Features

Line Highlighting

You can highlight specific lines in the code block by passing the `highlight-lines` parameter to the `zebraw` function. The `highlight-lines` parameter can be a single line number or an array of line numbers.

```
#zebraw(
  // Single line number:
  highlight-lines: 2,
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)

#zebraw(
  // Array of line numbers:
  highlight-lines: (6, 7) + range(9, 15),
  ```typ
  = Fibonacci sequence
  The Fibonacci sequence is defined through the
  recurrence relation $F_n = F_(n-1) + F_(n-2)$.
  It can also be expressed in _closed form:_

  $ F_n = round(1 / sqrt(5) phi.alt^n), quad
    phi.alt = (1 + sqrt(5)) / 2 $

  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = (
    if n ≤ 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  )

  The first #count numbers of the sequence are:

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
)
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

```
1 = Fibonacci sequence
2 The Fibonacci sequence is defined through the
3 recurrence relation $F_n = F_(n-1) + F_(n-2)$.
4 It can also be expressed in _closed form:_
5
6 $ F_n = round(1 / sqrt(5) phi.alt^n), quad
7   phi.alt = (1 + sqrt(5)) / 2 $
8
9 #let count = 8
10 #let nums = range(1, count + 1)
11 #let fib(n) = (
12   if n ≤ 2 { 1 }
13   else { fib(n - 1) + fib(n - 2) }
14 )
15
16 The first #count numbers of the sequence are:
17
18 #align(center, table(
19   columns: count,
20   ..nums.map(n => $F_#n$),
21   ..nums.map(n => str(fib(n))),
22 ))
```

Comment

You can add comments to the highlighted lines by passing an array of line numbers and comments to the `highlight-lines` parameter.

```
#zebraw(
  highlight-lines: (
    (1, [The Fibonacci sequence is defined through the recurrence relation $F_n = F_(n-1) + F_(n-2)$\
    It can also be expressed in _closed form:_ $ F_n = round(1 / sqrt(5) phi.alt^n), quad
    phi.alt = (1 + sqrt(5)) / 2 $]),
    // Passing a range of line numbers in the array should begin with `..`
    ..range(9, 14),
    (13, [The first \#count numbers of the sequence.]),
  ),
  ```typ
  = Fibonacci sequence
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = (
    if n ≤ 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  )

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
)
```

```
1 = Fibonacci sequence
  > The Fibonacci sequence is defined through the recurrence relation
  $F_n = F_{n-1} + F_{n-2}$
  It can also be expressed in closed form:
  $$F_n = \left\lfloor \frac{1}{\sqrt{5}} \phi^n \right\rceil, \quad \phi = \frac{1+\sqrt{5}}{2}$$
2 #let count = 8
3 #let nums = range(1, count + 1)
4 #let fib(n) = (
5   if n ≤ 2 { 1 }
6   else { fib(n - 1) + fib(n - 2) }
7 )
8
9 #align(center, table(
10   columns: count,
11   ..nums.map(n => $F_#n$),
12   ..nums.map(n => str(fib(n))),
13 ))
  > The first #count numbers of the sequence.
```

Comments can begin with a flag, which is `">"` by default. You can change the flag by passing the `comment-flag` parameter to the `zebraw` function:

```
#zebraw(
  highlight-lines: (
    // Comments can only be passed when highlight-lines is an array, so at the end of the single element array, a comma is needed.
    (6, [The Fibonacci sequence is defined through the recurrence relation $F_n = F_(n-1) + F_(n-2)$]),
  ),
  comment-flag: "⇝",
  ```typ
  = Fibonacci sequence
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = (
    if n ≤ 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  )

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
)
```

```
1 = Fibonacci sequence
2 #let count = 8
3 #let nums = range(1, count + 1)
4 #let fib(n) = (
5   if n ≤ 2 { 1 }
6   else { fib(n - 1) + fib(n - 2) }
    ⇝ The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$
7 )
8
9 #align(center, table(
10   columns: count,
11   ..nums.map(n => $F_#n$),
12   ..nums.map(n => str(fib(n))),
13 ))
```

To disable the flag feature, pass `""` to the `comment-flag` parameter (the indentation of the comment will be disabled as well):

```
#zebraw(
  highlight-lines: (
    (6, [The Fibonacci sequence is defined through the recurrence relation $F_n = F_(n-1) + F_(n-2)$]),
  ),
  comment-flag: "",
  ```typ
  = Fibonacci sequence
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = (
    if n ≤ 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  )

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
)
```

```
1 = Fibonacci sequence
2 #let count = 8
3 #let nums = range(1, count + 1)
4 #let fib(n) = (
5   if n ≤ 2 { 1 }
6   else { fib(n - 1) + fib(n - 2) }
  The Fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$
7 )
8
9 #align(center, table(
10   columns: count,
11   ..nums.map(n => $F_#n$),
12   ..nums.map(n => str(fib(n))),
13 ))
```

Header and Footer

Usually, the comments passing by a dictionary of line numbers and comments are used to add a header or footer to the code block:

```
#zebraw(
  highlight-lines: (
    (header: [*Fibonacci sequence*]),
    ..range(8, 13),
    // Numbers can be passed as a string in the dictionary, but it's too ugly.
    ("12": [The first \#count numbers of the sequence.]),
    (footer: [The fibonacci sequence is defined through the recurrence relation $F_n = F_(n-1) + F_(n-2)$]),
  ),
  ```typ
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = (
    if n ≤ 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  )

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
)
```

Fibonacci sequence

```
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = (
4   if n ≤ 2 { 1 }
5   else { fib(n - 1) + fib(n - 2) }
6 )
7
8 #align(center, table(
9   columns: count,
10   ..nums.map(n => $F_#n$),
11   ..nums.map(n => str(fib(n))),
12 ))
  > The first #count numbers of the sequence.
```

The fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$

Or you can use `header` and `footer` parameters to add a header or footer to the code block:

```
#zebraw(
  highlight-lines: (
    ..range(8, 13),
    (12, [The first \#count numbers of the sequence.]),
  ),
  header: [*Fibonacci sequence*],
  ```typ
  #let count = 8
  #let nums = range(1, count + 1)
  #let fib(n) = (
    if n ≤ 2 { 1 }
    else { fib(n - 1) + fib(n - 2) }
  )

  #align(center, table(
    columns: count,
    ..nums.map(n => $F_#n$),
    ..nums.map(n => str(fib(n))),
  ))
  ```
  ,
  footer: [The fibonacci sequence is defined through the recurrence relation $F_n = F_(n-1) + F_(n-2)$],
)
```

Fibonacci sequence

```
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = (
4   if n ≤ 2 { 1 }
5   else { fib(n - 1) + fib(n - 2) }
6 )
7
8 #align(center, table(
9   columns: count,
10   ..nums.map(n => $F_#n$),
11   ..nums.map(n => str(fib(n))),
12 ))
  > The first #count numbers of the sequence.
```

The fibonacci sequence is defined through the recurrence relation $F_n = F_{n-1} + F_{n-2}$

Language Tab

If `lang` is set to `true`, then there will be a language tab on the top right corner of the code block:

```
#zebraw(
  lang: true,
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

typst

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

Copyable

If `copyable` is set to `true`, line numbers will not be copied when copying exported code.

```
#zebraw(
  copyable: true,
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

For comparison:

`copyable: false`

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

`copyable: true`

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

However when a code block is `copyable`, it won't be able to cross page. Only line numbers will be excluded for being selected.

Customization

There are 3 ways to customize code blocks in your document:

- Manually render some specific blocks by `#zebraw()` function and passing parameters to it.
- By passing parameters to `#show: zebraw.with()` will affect every raw block after the `#show` rule, **except** blocks created manually by `#zebraw()` function.
- By passing parameters to `#show: zebraw-init.with()` will affect every raw block after the `#show` rule, **including** blocks created manually by `#zebraw()` function. By using `zebraw-init` without any parameters, the values will be reset to default.

Inset

Customize the inset of each line by passing a `dictionary` to the `inset` parameter:

```
#zebraw(
  inset: (top: 6pt, bottom: 6pt),
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

Colors

Customize the background color by passing a `color` or an `array` of `colors` to the `background-color` parameter.

```
#zebraw(
  background-color: luma(235),
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
    ,
)

#zebraw(
  background-color: (luma(235),
luma(245), luma(255)),
  ```typ
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
    ,
)
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

Customize the highlight color by passing a `color` to the `highlight-color` parameter:

```
#zebraw(
  highlight-lines: 1,
  highlight-color: blue.lighten(90%),
  ```text
  I'm so blue!
              -- George III
  ```
    ,
)
```

```
1 I'm so blue!
2               -- George III
```

Customize the comments' background color by passing a `color` to the `comment-color` parameter:

```
#zebraw(
  highlight-lines: (
    (2, "auto indent!"),
  ),
  comment-color: yellow.lighten(90%),
  ```text
  I'm so blue!
              -- George III
  I'm not.
              -- Hamilton
  ```
    ,
)
```

```
1 I'm so blue!
2               -- George III
                > auto indent!
3 I'm not.
4               -- Hamilton
```

Customize the language tab's background color by passing a `color` to the `lang-color` parameter.

```
#zebraw(
  lang: true,
  lang-color: teal,
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

typst

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

Font

To customize the arguments of comments' font and the language tab's font, pass a dictionary to `comment-font-args` parameter and `lang-font-args` parameter.

Language tab will be rendered as comments if nothing is passed.

```
#zebraw(
  highlight-lines: (
    (2, "columns ... "),
  ),
  lang: true,
  comment-color: white,
  comment-font-args: (
    font: "IBM Plex Sans",
    style: "italic"
  ),
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

typst

```
1 #grid(
2   columns: (1fr, 1fr),
    > columns...
3   [Hello], [world!],
4 )
```

```
#zebraw(
  highlight-lines: (
    (2, "columns ... "),
  ),
  lang: true,
  lang-color: eastern,
  lang-font-args: (
    font: "libertinus serif",
    weight: "bold",
    fill: white,
  ),
  comment-font-args: (
    font: "IBM Plex Sans",
    style: "italic"
  ),
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

typst

```
1 #grid(
2   columns: (1fr, 1fr),
    > columns...
3   [Hello], [world!],
4 )
```

Extend

Extend at vertical is enabled at default. When there's header or footer it will be automatically disabled.

```
#zebraw(
  extend: false,
  ```typst
  #grid(
    columns: (1fr, 1fr),
    [Hello], [world!],
  )
  ```
)
```

```
1 #grid(
2   columns: (1fr, 1fr),
3   [Hello], [world!],
4 )
```

Documentation

The default value of most parameters are none for it will use the default value in `zebraw-init`.

- `zebraw-init`
- `zebraw()`

`zebraw-init`

Initialize the `zebraw` block in global.

Parameters

`zebraw-init`

`inset` dictionary
`background-color` `none` | `auto` | `array`,
`highlight-color` `none`,
`comment-color` `none`,
`lang-color` `none`,
`comment-flag` `string` `content`,
`lang` `boolean`,
`comment-prefix-args` dictionary,
`lang-font-args` dictionary,
`extend` `boolean`,
`numbering` `boolean`,
`body` `content`
) → `content`

`inset` dictionary

The inset of each line.

Default: `(top: 0.3em, right: 0.3em, bottom: 0.3em, left: 0.3em)`

`background-color` `none` | `auto` | `array`

The background color of the block and normal lines.

Default: `luma(245)`

`highlight-color` `none` | `auto`

The background color of the highlighted lines.

Default: `rgb("#fea205").lighten(70%)`

`comment-color` `none` | `auto`

The background color of the comments. When it's set to `none`, it will be rendered in a lightened `highlight-color`.

Default: `none`

`lang-color` `none` | `auto`

The background color of the language tag. The color is set to `none` at default and it will be rendered in comments' color.

Default: `none`

`comment-flag` `string` | `content`

The flag at the beginning of comments.

Default: `";"`

`lang` `boolean`

Whether to show the language tag.

Default: `true`

`comment-prefix-args` dictionary

The arguments passed to comments' font.

Default: `(:)`

`lang-font-args` dictionary

The arguments passed to the language tab's font.

Default: `(:)`

`extend` `boolean`

Whether to extend the vertical spacing.

Default: `true`

`copyable` `boolean`

Whether to enable "copyable" feature.

Default: `false`

`body` `content`

The body

`zebraw`

Block of code with highlighted lines and comments.

```
zebraw(
  highlight-lines: (
    10,
    (9, "10 or 12345678923"),
    12,
  ),
  lang: true,
  header: "Calculate Fibonacci number using
recursive function",
  numbering: (
    n) => "Fibonacci_recursive(n: 12)" +
str(n)
  )
  if n == 0 {
    panic("It is negative", n);
  } else {
    match n {
      0 => panic("zero is not a right
argument to Fibonacci_recursive!"),
      1 | 2 => 1,
      3 => 2,
      _ => {
        Fibonacci_recursive(n: n - 1)
        + Fibonacci_recursive(n: n - 2),
        n > 10 or 12345678923
      }
    }
  }
}
```

```
Calculate Fibonacci number using
recursive function
1 pub fn Fibonacci_recursive(n:
u32) -> u64 {
2     if n == 0 {
3         panic("It is negative",
n);
4     }
5     match n {
6         0 => panic("zero is not
a right argument to
Fibonacci_recursive!"),
7         1 | 2 => 1,
8         3 => 2,
9         _ => {
10        Fibonacci_recursive(n: n - 1)
11        + Fibonacci_recursive(n: n - 2),
12        n > 10 or 12345678923
13    }
14    }
15 }
```

Parameters

`zebraw`

`highlight-lines`: `array` | `int`,
`header`: `string` | `content`,
`footer`: `string` | `content`,
`inset`: `dictionary`,
`background-color` `none` | `auto` | `array`,
`highlight-color` `none` | `auto`,
`comment-color` `none` | `auto`,
`lang-color` `none` | `auto`,
`comment-flag`: `string` | `content`,
`lang`: `boolean`,
`comment-font-args`: `dictionary`,
`lang-font-args`: `dictionary`,
`extend`: `boolean`,
`copyable`: `boolean`,
`body`: `content`
) → `content`

`highlight-lines` `array` | `int`

Lines to highlight or comments to show.

```
Fibonacci sequence
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = {
4   if n <= 2 { 1 }
5   else { fib(n - 1) + fib(n -
2) }
6 }
7
8 #align(center, table(
9   columns: count,
10  ..nums.map(n => $F_#n$),
11  ..nums.map(n => str(fib(n)))
12 ))
```

```
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = {
4   if n <= 2 { 1 }
5   else { fib(n - 1) + fib(n -
2) }
6 }
7
8 #align(center, table(
9   columns: count,
10  ..nums.map(n => $F_#n$),
11  ..nums.map(n => str(fib(n)))
12 ))
```

Default: `none`

`header` `string` | `content`

The header of the code block.

Default: `none`

`footer` `string` | `content`

The footer of the code block.

Default: `none`

`inset` dictionary

The inset of each line.

```
Fibonacci sequence
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = {
4   if n <= 2 { 1 }
5   else { fib(n - 1) + fib(n -
2) }
6 }
7
8 #align(center, table(
9   columns: count,
10  ..nums.map(n => $F_#n$),
11  ..nums.map(n => str(fib(n)))
12 ))
```

```
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = {
4   if n <= 2 { 1 }
5   else { fib(n - 1) + fib(n -
2) }
6 }
7
8 #align(center, table(
9   columns: count,
10  ..nums.map(n => $F_#n$),
11  ..nums.map(n => str(fib(n)))
12 ))
```

Default: `none`

`background-color` `none` | `auto` | `array`

The background color of the block and normal lines.

```
Fibonacci sequence
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = {
4   if n <= 2 { 1 }
5   else { fib(n - 1) + fib(n -
2) }
6 }
7
8 #align(center, table(
9   columns: count,
10  ..nums.map(n => $F_#n$),
11  ..nums.map(n => str(fib(n)))
12 ))
```

```
1 #let count = 8
2 #let nums = range(1, count + 1)
3 #let fib(n) = {
4   if n <= 2 { 1 }
5   else { fib(n - 1) + fib(n -
2) }
6 }
7
8 #align(center, table(
9   columns: count,
10  ..nums.map(n => $F_#n$),
11  ..nums.map(n => str(fib(n)))
12 ))
```

Default: `none`

`highlight-color` `none` | `auto`

The background color of the highlighted lines.

Default: `none`

`comment-color` `none` | `auto`

The background color of the comments. The color is set to `none` at default and it will be rendered in a lightened `highlight-color`.

```
Fibonacci sequence
1 #zebraw(
2   highlight-color: yellow.lighten(60%),
3   comment-color: blue.lighten(90%),
4   ) // This function is defined
through the recurrence relation
5 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
6 #let nums = range(1, count + 1)
7 #let fib(n) = {
8   if n <= 2 { 1 }
9   else { fib(n - 1) + fib(n -
2) }
10 }
11
12 #align(center, table(
13   columns: count,
14   ..nums.map(n => $F_#n$),
15   ..nums.map(n => str(fib(n)))
16 ))
17 ...
```

```
1 #zebraw(
2   highlight-color: yellow.lighten(60%),
3   comment-color: blue.lighten(90%),
4   ) // This function is defined
through the recurrence relation
5 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
6 #let nums = range(1, count + 1)
7 #let fib(n) = {
8   if n <= 2 { 1 }
9   else { fib(n - 1) + fib(n -
2) }
10 }
11
12 #align(center, table(
13   columns: count,
14   ..nums.map(n => $F_#n$),
15   ..nums.map(n => str(fib(n)))
16 ))
17 ...
```

Default: `none`

`lang-color` `none` | `auto`

The background color of the language tab. The color is set to `none` at default and it will be rendered in comments' color.

```
Fibonacci sequence
1 #zebraw(
2   lang-color: eastern,
3   lang-font-args: (
4     font: "Noto Serif CJK
JP",
5     weight: "bold",
6     fill: white
7   ),
8   ) // This function is defined
through the recurrence relation
9 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
10 #let nums = range(1, count + 1)
11 #let fib(n) = {
12   if n <= 2 { 1 }
13   else { fib(n - 1) + fib(n -
2) }
14 }
15
16 #align(center, table(
17   columns: count,
18   ..nums.map(n => $F_#n$),
19   ..nums.map(n => str(fib(n)))
20 ))
21 ...
```

```
1 #zebraw(
2   lang-color: eastern,
3   lang-font-args: (
4     font: "Noto Serif CJK
JP",
5     weight: "bold",
6     fill: white
7   ),
8   ) // This function is defined
through the recurrence relation
9 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
10 #let nums = range(1, count + 1)
11 #let fib(n) = {
12   if n <= 2 { 1 }
13   else { fib(n - 1) + fib(n -
2) }
14 }
15
16 #align(center, table(
17   columns: count,
18   ..nums.map(n => $F_#n$),
19   ..nums.map(n => str(fib(n)))
20 ))
21 ...
```

Default: `none`

`comment-flag` `string` | `content`

The flag at the beginning of comments. The indentation of codes will be rendered before the flag. When the flag is set to `""`, the indentation before the flag will be disabled as well.

```
Fibonacci sequence
1 #zebraw(
2   comment-flag: "-",
3   highlight-lines: (
4     (9, "The Fibonacci sequence is
defined through the recurrence relation
$F_n = F_{n-1} + F_{n-2}$"),
5   ) // This function is defined
through the recurrence relation
6 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
7 #let nums = range(1, count + 1)
8 #let fib(n) = {
9   if n <= 2 { 1 }
10  else { fib(n - 1) + fib(n -
2) }
11 }
12
13 #align(center, table(
14   columns: count,
15   ..nums.map(n => $F_#n$),
16   ..nums.map(n => str(fib(n)))
17 ))
18 ...
```

```
1 #zebraw(
2   comment-flag: "-",
3   highlight-lines: (
4     (9, "The Fibonacci sequence is
defined through the recurrence relation
$F_n = F_{n-1} + F_{n-2}$"),
5   ) // This function is defined
through the recurrence relation
6 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
7 #let nums = range(1, count + 1)
8 #let fib(n) = {
9   if n <= 2 { 1 }
10  else { fib(n - 1) + fib(n -
2) }
11 }
12
13 #align(center, table(
14   columns: count,
15   ..nums.map(n => $F_#n$),
16   ..nums.map(n => str(fib(n)))
17 ))
18 ...
```

Default: `none`

`lang` `boolean`

Whether to show the language tab on the top-right corner of the code block.

```
Fibonacci sequence
1 #zebraw(
2   lang: true,
3   ) // This function is defined
through the recurrence relation
4 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
5 #let nums = range(1, count + 1)
6 #let fib(n) = {
7   if n <= 2 { 1 }
8   else { fib(n - 1) + fib(n -
2) }
9 }
10
11 #align(center, table(
12   columns: count,
13   ..nums.map(n => $F_#n$),
14   ..nums.map(n => str(fib(n)))
15 ))
16 ...
```

```
1 #zebraw(
2   lang: true,
3   ) // This function is defined
through the recurrence relation
4 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
5 #let nums = range(1, count + 1)
6 #let fib(n) = {
7   if n <= 2 { 1 }
8   else { fib(n - 1) + fib(n -
2) }
9 }
10
11 #align(center, table(
12   columns: count,
13   ..nums.map(n => $F_#n$),
14   ..nums.map(n => str(fib(n)))
15 ))
16 ...
```

Default: `none`

`comment-font-args` dictionary

The arguments passed to comments' font.

Default: `none`

`lang-font-args` dictionary

The arguments passed to the language tab's font.

```
Fibonacci sequence
1 #zebraw(
2   lang: true,
3   comment-font-args: (font: "IBM Plex
Serif", style: "italic"),
4   lang-font-args: (font: "IBM Plex Sans",
weight: "bold"),
5   highlight-lines: (
6     (9, "The Fibonacci sequence is
defined through the recurrence relation
$F_n = F_{n-1} + F_{n-2}$"),
7     (11, "The first #count numbers of
the sequence."),
8   ) // This function is defined
through the recurrence relation
9 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
10 #let nums = range(1, count + 1)
11 #let fib(n) = {
12   if n <= 2 { 1 }
13   else { fib(n - 1) + fib(n -
2) }
14 }
15
16 #align(center, table(
17   columns: count,
18   ..nums.map(n => $F_#n$),
19   ..nums.map(n => str(fib(n)))
20 ))
21 ...
```

```
1 #zebraw(
2   lang: true,
3   comment-font-args: (font: "IBM Plex
Serif", style: "italic"),
4   lang-font-args: (font: "IBM Plex Sans",
weight: "bold"),
5   highlight-lines: (
6     (9, "The Fibonacci sequence is
defined through the recurrence relation
$F_n = F_{n-1} + F_{n-2}$"),
7     (11, "The first #count numbers of
the sequence."),
8   ) // This function is defined
through the recurrence relation
9 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
10 #let nums = range(1, count + 1)
11 #let fib(n) = {
12   if n <= 2 { 1 }
13   else { fib(n - 1) + fib(n -
2) }
14 }
15
16 #align(center, table(
17   columns: count,
18   ..nums.map(n => $F_#n$),
19   ..nums.map(n => str(fib(n)))
20 ))
21 ...
```

Default: `none`

`extend` `boolean`

Whether to extend the vertical spacing.

```
Fibonacci sequence
1 #zebraw(
2   extend: false,
3   ) // This function is defined
through the recurrence relation
4 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
5 #let nums = range(1, count + 1)
6 #let fib(n) = {
7   if n <= 2 { 1 }
8   else { fib(n - 1) + fib(n -
2) }
9 }
10
11 #align(center, table(
12   columns: count,
13   ..nums.map(n => $F_#n$),
14   ..nums.map(n => str(fib(n)))
15 ))
16 ...
```

```
1 #zebraw(
2   extend: false,
3   ) // This function is defined
through the recurrence relation
4 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
5 #let nums = range(1, count + 1)
6 #let fib(n) = {
7   if n <= 2 { 1 }
8   else { fib(n - 1) + fib(n -
2) }
9 }
10
11 #align(center, table(
12   columns: count,
13   ..nums.map(n => $F_#n$),
14   ..nums.map(n => str(fib(n)))
15 ))
16 ...
```

Default: `true`

`copyable` `boolean`

Whether to enable "copyable" feature.

If copyable feature is enabled, lines of code are displayed together in a block to allow copying code in an export. However, it uses a lot of `measure` functions, which may slow down compilation efficiency. Also when this feature is enabled, the code block **cannot** be displayed across pages.

```
Fibonacci sequence
1 #zebraw(
2   copyable: true,
3   ) // This function is defined
through the recurrence relation
4 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
5 #let nums = range(1, count + 1)
6 #let fib(n) = {
7   if n <= 2 { 1 }
8   else { fib(n - 1) + fib(n -
2) }
9 }
10
11 #align(center, table(
12   columns: count,
13   ..nums.map(n => $F_#n$),
14   ..nums.map(n => str(fib(n)))
15 ))
16 ...
```

```
1 #zebraw(
2   copyable: true,
3   ) // This function is defined
through the recurrence relation
4 #let count = 8 // The Fibonacci
sequence $F_n = F_{n-1} + F_{n-2}$,
5 #let nums = range(1, count + 1)
6 #let fib(n) = {
7   if n <= 2 { 1 }
8   else { fib(n - 1) + fib(n -
2) }
9 }
10
11 #align(center, table(
12   columns: count,
13   ..nums.map(n => $F_#n$),
14   ..nums.map(n => str(fib(n)))
15 ))
16 ...
```

Default: `none`

`body` `content`

The body.

Example

rust

Calculate Fibonacci number using reccursive function

```rust
pub fn fibonacci_reccursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
        > to avoid negative numbers
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_reccursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_reccursive(n - 1) + fibonacci_reccursive(n - 2),
        > 50 => 12586269025
    }
}
```

rust

```rust
pub fn fibonacci_reccursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
        > negative numbers are not allowed
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_reccursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_reccursive(n - 1) + fibonacci_reccursive(n - 2),
    }
}
```

rust

```rust
pub fn fibonacci_reccursive(n: i32) -> u64 {
    if n < 0 {
        panic!("{} is negative!", n);
        > negative numbers are not allowed
    }
    match n {
        0 => panic!("zero is not a right argument to fibonacci_reccursive()!"),
        1 | 2 => 1,
        3 => 2,
        _ => fibonacci_reccursive(n - 1) + fibonacci_reccursive(n - 2),
    }
}
```